KU LEUVEN

Geavanceerde Computerarchitectuur

GPU-versnelde aerodynamische simulatie van FPV-frames met de Lattice Boltzmann Method (D2Q9/CUDA)

Isaak Vervenne Sander De Stercke

Mei 2026

Samenvatting

In dit project werd een GPU-versnelde 2D computational fluid dynamics (CFD) solver gebouwd op basis van de *Lattice Boltzmann* methode (D2Q9). Het doel is het aerodynamisch gedrag van FPV-drones versneld te simuleren op de GPU. De solver is geïmplementeerd in drie varianten: CUDA, CPU single-core en CPU multi-core (OpenMP), en ingebed in een volledige pipeline: van 3D STL-model van de drone tot de animatie ervan, aangestuurd via een webinterface. De GPU-implementatie is $\approx 80\times$ sneller dan de single-core referentie en $\approx 11\times$ sneller dan de 28-thread OpenMP-versie.

Inhoudsopgave

1 Inleiding en Probleemstelling **2**

2 Theoretisch Kader: Lattice Boltzmann Method **2**

2.1 D2Q9-model . . . 2

2.2 Evenwichtsfunctie en macroscopische grootheden . . . 3

2.3 Fysische parameters en stabiliteit . . . 3

3 Implementatie **3**

3.1 CUDA-kernel: `lbm_step_pull` . . . 4

3.2 CPU-varianten . . . 5

3.3 STL-naar-masker pipeline (`stl_to_mask.py`) . . . 5

3.4 I/O-optimalisatie: binair en asynchroon . . . 6

3.5 Animatie en UI . . . 6

4 Resultaten en evaluatie **7**

4.1 Prestaties . . . 7

4.2 Diagnostiek . . . 7

4.3 STL multi-view resultaten . . . 7

5 Tekortkomingen en Toekomstig Werk **9**

6 Besluit **9**

1 Inleiding en Probleemstelling

FPV-dronepiloten optimaliseren hun frames voortdurend: elke gram materiaal en elke millimeter profiel beïnvloeden het aerodynamische gedrag en daarmee de vliegprestaties. Klassieke windtunneltests zijn te duur voor hobbyisten zoals wij, terwijl commerciële CFD-pakketten te complex zijn voor snelle iteraties. Wij bouwen daarom een GPU-versnelde solver die in enkele minuten een volledig snelheidsveld genereert voor een willekeurig 3D STL-model.

Waarom de GPU?

De Lattice Boltzmann Method (LBM) heeft als voordeel dat de berekening per iteratie volledig lokaal is: de nieuwe toestand van een cel hangt enkel af van zijn directe buren. Er zijn dus geen globale afhankelijkheden, waardoor berekeningen niet op elkaar moeten wachten.

Voor de architectuur betekent dit dat we een *perfect data-parallel* probleem hebben. Ons rooster van 2048×1024 bestaat uit ruim twee miljoen cellen. Omdat de cellen onafhankelijk van elkaar geüpdatet kunnen worden, wijzen we simpelweg elke cel toe aan een eigen thread op de GPU. Hierdoor kunnen we het grote aantal GPU cores tegelijkertijd en continu aan het werk zetten, zonder dat de prestaties inzakken door synchronisatie-overhead.

2 Theoretisch Kader: Lattice Boltzmann Method

2.1 D2Q9-model

Het D2Q9-model discretiseert de ruimte naar een 2D rooster met 9 snelheidsvectoren per cel (inclusief de rustvector). De distributiefuncties $f_i(\mathbf{x}, t)$ beschrijven de deeltjesdichtheid in de richting i. Elke iteratie bestaat uit twee stappen:

1. **Streaming (pull-schema):** distributies worden vanuit de buurcel in tegengestelde richting gelezen:
$$f_i^*(\mathbf{x}) = f_i(\mathbf{x} - \mathbf{c}_i)$$

2. **Collision (BGK-operator):** relaxatie naar het lokaal evenwicht:
$$f_i^{\text{new}} = f_i^* - \omega(f_i^* - f_i^{\text{eq}})$$

2.2 Evenwichtsfunctie en macroscopische grootheden

De evenwichtsverdeling is:

$$f_i^{\text{eq}} = w_i \rho \left(1 + 3(\mathbf{c}_i \cdot \mathbf{u}) + \tfrac{9}{2}(\mathbf{c}_i \cdot \mathbf{u})^2 - \tfrac{3}{2}\|\mathbf{u}\|^2\right)$$

met roostergewichten $w_0 = \frac{4}{9}$, $w_{1\text{–}4} = \frac{1}{9}$ en $w_{5\text{–}8} = \frac{1}{36}$. De roostergewichten representeren het gewicht dat aan een specifieke richting wordt toegekend. De macroscopische dichtheid ρ en snelheid $\mathbf{u}$ volgen uit de momenten:

$$\rho = \sum_i f_i, \qquad \rho\mathbf{u} = \sum_i f_i \mathbf{c}_i.$$

2.3 Fysische parameters en stabiliteit

Relaxatieparameter ω: bepaalt de kinematische viscositeit:

$$\nu_{\text{LBM}} = c_s^2 \left(\frac{1}{\omega} - \frac{1}{2}\right), \quad c_s^2 = \tfrac{1}{3}.$$

Gekozen waarde: $\omega = \mathbf{1.8}$ ($\nu \approx 0.0185$). Het BGK-model wordt numeriek instabiel voor $\omega \to 2$.

Mach-getal: Om incompressibiliteit te respecteren, dient $Ma = u_{\text{in}}/c_s \ll 0.3$. Met $u_{\text{in}} = 0.04$ geldt:

$$Ma = 0.04\sqrt{3} \approx 0.069$$

Reynoldsgetal: $Re = u_{\text{in}} \cdot L/\nu_{\text{LBM}}$. Door ω te verhogen (lagere viscositeit) kan een turbulenter stromingsprofiel worden gesimuleerd. Met de gekozen waarde voor $\omega = 1.8$ komt dit overeen met een fluïdum met een lage viscositeit, zoals lucht.

3 Implementatie

De volledige coderepository bevat vier uitvoerbare programma's en één Python/web-pipeline. De bestandsstructuur is als volgt:

Bestand	Omschrijving
`fpv_cfd_stl.cu`	CUDA LBM-solver met extern masker
`fpv_cfd_cpu_single.cpp`	CPU single-core referentie
`fpv_cfd_cpu_multi.cpp`	CPU multi-core (OpenMP)
`python/stl_to_mask.py`	STL → 2D obstakelmasker (Numba JIT)
`python/animate_cfd.py`	Binaire frames → MP4 (multiprocessing)
`web/app.py`	FastAPI webinterface + WebSocket pipeline

3.1 CUDA-kernel: `lbm_step_pull`

Pull-streaming en pointer swapping

De kernel voert streaming en collision in één keer uit via een *pull-schema*: elke thread leest enkel uit zijn buurcellen en schrijft er niet naar. Eén thread is dus verantwoordelijk voor zijn eigen cel en de operaties ervan. Dit elimineert write-conflicten en heeft geen gedeeld geheugen nodig. Na elke iteratie wisselen de pointers d_{fA} (input) en d_{fB} (output) van rol via *std::swap*, zodat geen extra *cudaMemcpy* vereist is. Dit voorkomt dat wanneer de waarde van een cel wordt aangepast, een buurcel die de originele waarde nog nodig heeft, een foutieve waarde inleest.

Structure of Arrays (SoA) voor memory coalescing

De distributiefuncties worden opgeslagen in SoA-formaat: $f[i * cells + idx]$ in plaats van AoS $f[idx * Q + i]$. Concreet betekent dit dat per cel niet elke richting wordt bijgehouden, maar dat een bepaalde richting wordt bijgehouden voor alle cellen. Dit zorgt ervoor dat de geheugenadressen per specifieke richting dicht bij elkaar worden geplaatst in het geheugen. Bij een warp van 32 threads die cel $0 \ldots 31$ verwerken, lezen alle threads aaneengesloten geheugenadressen voor richting i. Dit is de definitie van **memory coalescing** op CUDA.

```
if (obstacle[idx]) {
    for (int i = 0; i < Q; ++i)
        fOut[i*cells + idx] = fIn[OPP[i]*cells + idx]; // SoA: coalesced
    return;
}
```

Listing 1: SoA bounce-back op een obstacle-cel (uit `fpv_cfd_stl.cu:31`)

Randvoorwaarden

- **Invloei:** equilibriumverdeling met vaste instroomsnelheid u_{in} (richting instelbaar via parameter).
- **Uitvloei:** zero-gradient (kopie van aangrenzende cel).
- **Wanden:** elastische botsing (snelheidsomkering).
- **Obstakels:** botsing op basis van het binaire masker van de STL.

NaN-bescherming

Na elke collision controleert de kernel of het resultaat eindig is:

```
float out = fLoc[i] - omega * (fLoc[i] - fe);
fOut[i*cells + idx] = isfinite(out) ? out : fe;
```

Listing 2: NaN-bescherming in de collision-stap (fpv_cfd_stl.cu:76)

Dit voorkomt dat een instabiele cel de hele simulatie laat exploderen.

3.2 CPU-varianten

De CPU-versies gebruiken dezelfde numerieke update als bij de CUDA-kernel:

- **Single-core:** sequentiële dubbele lus over x, y (geneste loop).
- **Multi-core:** Parallelliseert de berekening over alle beschikbare CPU-threads met behulp van OpenMP. Door de constructie "`#pragma omp parallel for collapse(2)`" worden de geneste lussen geparalleliseerd en samengevoegd tot één datapoel, wat zorgt voor een goede werklastverdeling.

Beide varianten dienen als referentie om de speedup-meting uit te voeren.

3.3 STL-naar-masker pipeline (`stl_to_mask.py`)

1. Laad het binaire of ASCII STL-bestand in driehoeken.
2. Optioneel: Pas een 3D-rotatie en spiegeling toe vóór de projectie.
3. Projecteer op het gewenste vlak (top / front / side).
4. Pas een 2D-rotatie toe (`-heading-deg`) voor de gewenste invalshoek.
5. Rasteriseer het mesh van driehoeken naar een binair $N_x \times N_y$ masker (Numba JIT-gecompileerde rasterizer).
6. Schrijf het masker als sparse CSV (`x,y,occ`).

De JIT-compilatie reduceert de rasterisatietijd van $\approx 60\,\mathrm{s}$ (pure Python) naar enkele milliseconden voor een typisch frame-STL.

3.4 I/O-optimalisatie: binair en asynchroon

Probleem

Bij een grote simulatie (2048×1024, 100 000 iteraties, frame elke 100 iteraties) produceerde de oorspronkelijke CSV-export ≈ 35 MB per frame (≈ 35 GB totaal). De CPU moest $> 2 \times 10^6$ floats per frame naar ASCII converteren, wat de GPU volledig onderuit haalde: GPU-utilisatie zakte naar 6%.

Oplossing

Binaire dumps (`.bin`): de raw `float32`-array wordt direct via `fwrite` weggeschreven — geen conversie, 8.3 MB per frame (75% kleiner).

Asynchrone schrijfthread: Om te voorkomen dat de snelle GPU moet wachten op de tragere harde schijf, wordt er asynchroon gewerkt. Na elke GPU-synchronisatie kopieert `cudaMemcpy` de berekende snelheidsarray direct naar een buffer in het hoofdgeheugen (host). Vervolgens neemt een aparte CPU-thread (`std::thread`) het wegschrijven naar de schijf op zich, waardoor de GPU onmiddellijk kan starten met de volgende iteratiebatch.

```
CUDA_CHECK(cudaMemcpy(h_vel.data(), d_vel, velBytes, cudaMemcpyDeviceToHost));

if (writer_thread.joinable()) writer_thread.join(); // join na memcpy

std::vector<float> h_vel_copy = h_vel;
writer_thread = std::thread([path_copy, h_vel_copy]() {
    write_velocity_bin(path_copy, h_vel_copy); // fwrite raw binary
});
```

Listing 3: Overlapping compute en I/O (fpv_cfd_stl.cu:186–192)

De totale simulatietijd daalde van > 20 min naar < 5 min. Bijgevolg verdwenen crashes door geheugenuitputting volledig.

3.5 Animatie en UI

De Python-scripts in `python/` verzorgen post-processing:

- `animate_cfd.py`: Inladen van alle `.bin`-frames (`numpy.fromfile`) ; Parallel renderen van PNG-frames (`ProcessPoolExecutor`) ; Assemblage van een MP4-bestand (`imageio[pyav]`, libx264).
- `web/app.py`: FastAPI-backend met WebSocket-verbinding voor real-time voortgangsrapportering. De vier pipeline-stappen (compileren, masker genereren, CUDA-simulatie, MP4 renderen) worden asynchroon (`asyncio.Task`) uitgevoerd.

4 Resultaten en evaluatie

4.1 Prestaties

Implementatie	Runtime (s)	Speedup t.o.v. GPU
GPU CUDA	0.83	1.0×
CPU multi-core (28)	9.42	0.09× (11.3× trager)
CPU single-core	66.37	0.01× (79.7× trager)

Tabel 1: Runtimevergelijking bij gelijke configuratie: rooster 512×256, 12 000 iteraties, compiler `-O3 -arch=sm_75`.

De GPU is $\approx 80\times$ sneller dan de single-core referentie en $\approx 11\times$ sneller dan 28-thread OpenMP, wat de hoge parallelliseergraad van LBM bevestigt.

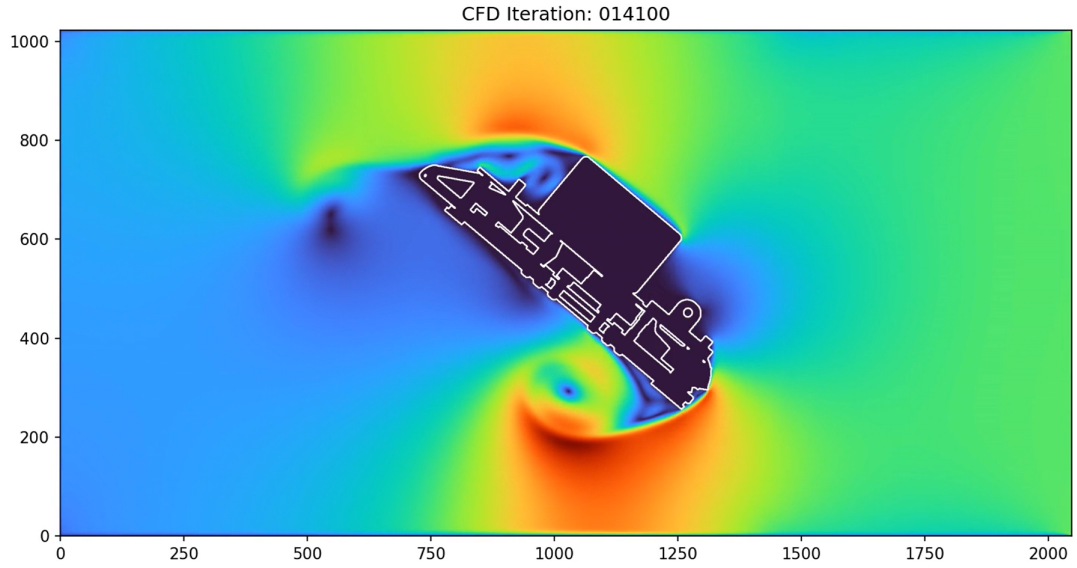
4.2 Diagnostiek

Op het einde van de simulatie (12 000 iteraties, 512×256):

- `nan count = 0` — numeriek stabiel.
- $v_{\max} = 0.0720$ ($Ma \approx 0.125 < 0.3$ ✓).
- $v_{\text{gem}} = 0.0347$.
- 124 069 / 131 072 cellen actief ($> 1 \times 10^{-6}$).

4.3 STL multi-view resultaten

De solver is getest met top-, front- en side-views van een echt FPV-frame STL. Door `-heading-deg` te variëren kan de invalshoek van de luchtstroom worden gesimuleerd zonder de solverarchitectuur te wijzigen. Animaties (MP4) zijn beschikbaar in de repository onder `code/images/`. Figuur 1 toont een representatief frame van de gesimuleerde luchtstroom rond het FPV-frame.



Figuur 1: Snelheidsveld rond een FPV-drone frame, berekend met de CUDA LBM-solver. De stroming gaat van rechts naar links; kleurintensiteit geeft de lokale snelheidsmagnitude weer. Duidelijk zichtbaar zijn de stagnatiezone aan de voorzijde en het turbulente zog achter het frame.

5 Tekortkomingen en Toekomstig Werk

BGK-instabiliteit bij hoge Re: het BGK-model wordt numeriek instabiel wanneer $\omega \to 2$ (lage viscositeit). Dit beperkt het simuleerbare Reynolds-getal. Het *Multiple Relaxation Time* (MRT) model of het *Two Relaxation Time* (TRT) model bieden betere stabiliteit bij hogere Reynolds-getallen.

2D-projectie verliest informatie: door het frame te projecteren op een 2D-vlak gaat er informatie verloren over uitsteeksels en interne structuren. Een 3D-uitbreiding naar het D3Q19-model zou een realistischere analyse bieden.

Enkelvoudige GPU (VRAM-limiet): grote roosters ($> 4096 \times 2048$) zijn beperkt door de beschikbare VRAM. Multi-GPU-uitbreiding via CUDA-aware MPI is mogelijk maar vereist domeindecompositie en halo-uitwisseling.

SSD I/O als bottleneck: ondanks de asynchrone binaire aanpak treedt er periodiek een "dirty-page-cache flush" op (Linux schrijft gebufferde pagina's weg), wat leidt tot korte GPU-stilstanden. Een snellere NVMe-schijf of in-memory bestandssysteem (tmpfs) zou dit elimineren.

Vaste gridresolutie: de solver gebruikt een uniform Cartesiaans rooster. Adaptieve verfijning (Adaptive Mesh Refinement) rond het obstakel zou nauwkeuriger resultaten geven met minder cellen.

6 Besluit

We hebben een volledige GPU-versnelde CFD-pipeline gebouwd, van 3D STL-import tot geanimeerde stroomlijnen in de browser. De CUDA LBM-solver haalt een speedup van $\approx 80\times$ ten opzichte van een single-core referentie bij een identieke numerieke update, wat het potentieel van GPU-computing voor dit type massief parallelle berekening aantoont. De voornaamste bottleneck bij hogere resoluties bleek niet de rekenkracht maar de I/O; dit is opgelost via binaire dumps en een asynchrone schrijfthread, waardoor de GPU-utilisatie nagenoeg constant op 100% blijft. Verdere optimalisatie van de I/O kan mogelijks tot nog efficiëntere resultaten leiden.